Assignment 1

Linear Regression Gradient Descents

How change in parameters affect the execution of the Algorithm

Xander Palermo

Missouri State University

CSC 537 – Deep Learning

Mukulika Ghosh

30 January 2026

Introduction

For this project, we were asked to develop code that will generate training and testing data for a Linear Regression model, as well craft our own Regression Model training and testing sequence. While I am unsure if my implementations are completely correct, I believe that this exercise was very helpful overall towards contributing to my understanding of the topic for future implementations.

Methodology

This project was broken into 4 separate files divided on the goal that each script accomplishes. Each are discussed in some detail below.

generator.py

This file is responsible for generating the 6,000 training samples and 2,000 testing samples as defined in the project requirements. The inputs are generated using the formula provided:

$$y = 4x_1 + 3x_2 + 2.5x_3 - x_4 + 0.5x_5 + \varepsilon\,, \qquad \varepsilon \sim \mathcal{N} \sim (0,1)$$

To generate the inputs of the samples, I used the recommended normal distribution provided in the NumPy library. I also used a set-seed of 0 to create consistent results and increase the repeatability of my experiments. The two sets of generated data are then saved to the *dataset* directory and labeled for their use case. They are saved as a csv, comma separated values, file such that each row contains each generated value of a feature and the last column (column 6) contains the target value that the generator calculated for those features.

gradient_descent.py

This file implements the Linear Regression algorithms for training and testing. For readability, many functions have been refactored to helper functions to keep all function declarations under 30 statements long.

Training Function

The training function begins with initializing all constant and counting variables it needs to execute the algorithm. The algorithm is then similarly executed as it is explained in the lectures. Every epoch begins with shuffling the data (user provided) if the user has enabled stochastic gradient descent (the *shuffle* parameter). The data is then divided into batches as also designated by the user in the *batch_size* parameter. Batches that are at the end of the epoch, and do not have enough samples to fill the entire batch, are left as partials and computing using however many samples are remaining. During each iteration, linear algebra is applied to adjust the weights using the following equation:

$$\Delta_{\overrightarrow{(w)}} = \alpha \times (\frac{1}{B} X^T (X\vec{w} - \vec{y}) + 2\lambda\vec{w})$$

This is such that B is the size of the batch, X is the matrix of features samples, w is a vector of the current value of the calculated weights, and y is the actual target value generated by *generator.py*. Lambda (λ) is a hyper-parameter set by the user that influences the amount of weight decay that takes place during each iteration. The larger the value of lambda, the stronger the weights are pulled back to 0. This weight calculation is also multiplied to a learning rate, alpha (α), that influences how much the weights can

change in each iteration. When lambda is equal to 0, this piece of the function can be ignored and can be said that there is no weight decay active. Statistics recorded during the iterations are discussed in the *logger.py* section. The algorithm has two ways of self-terminating. The first way is by calculating using the user provided parameter *tol.* This parameter represents epsilon (ε), which represents the amount of error that is allowed when checking the cost function. The cost function is evaluated using the following formula:

$$\mathcal{L}(w) = \frac{1}{2B}\left|\left|X\vec{w} - \vec{y}\right|\right|^2 + \lambda\left|\left|\vec{w}\right|\right|^2$$

When this value is calculated to be lower than the epsilon value, the algorithm can terminate, as it has reached a "good-enough" convergence. The second way that this algorithm can self-terminate is when it iterates past the user-defined number of maximum iterations. This is a safety put in place to ensure infinite iterations and do not occur. If the algorithm terminates in this manner, no proper convergence was found; implying that parameters may need to be changed.

Testing Function

This implementation of a testing function was not very strong, as it was a topic that I did not understand the process of from class. It functions in all the same ways the training function does during its algorithm, however it acts on a precalculated matrix of weights over iterations, performed on data not yet seen by the training algorithm. This provides data on how each iteration of weights minimizes the cost function.

logger.py

This small file creates a container class used to bundle various types of data needed for analysis of the regular regression's functioning over its iterations. The most important item that is logged in this class is *weights_data*. This is a matrix that maintains a column of each feature vector and each row depicts what its value was in any given iteration. It is thanks to this logger class that the testing function can retrace the steps taken by the training function so that the two can be ran asynchronously.

The logger class also records data of the loss formula over time, as well as its individual parts, being the Mean Squared Error and L2 Ridge. These are important sets of data that are used in plots in the experiments with different user-set parameters.

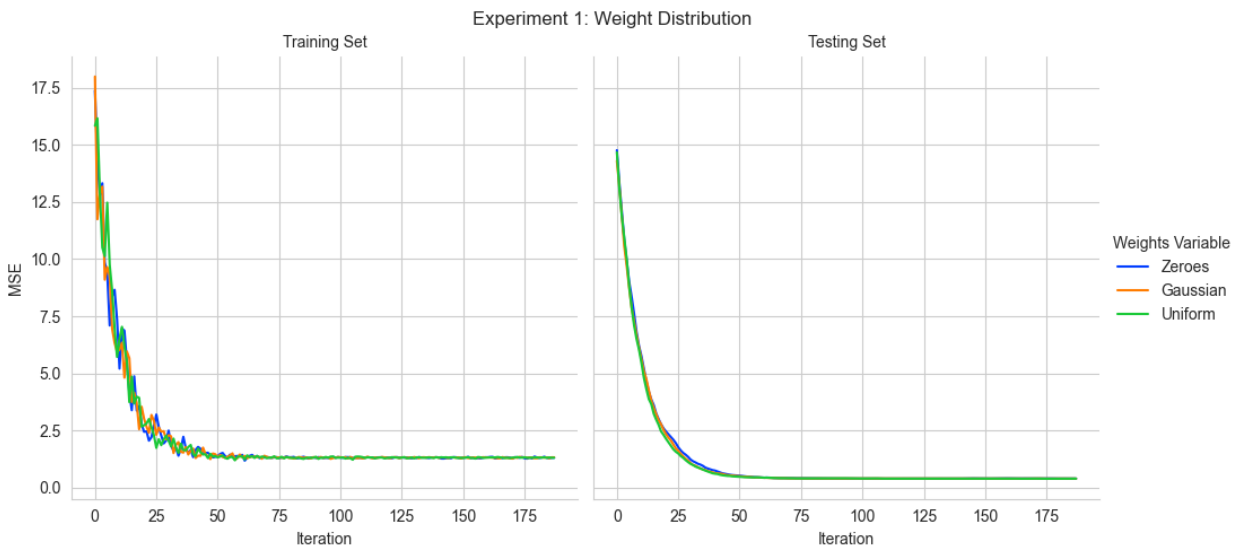
plotter.py

This file implements all the experiments that are specified in the assignment requirements. There are a few helper functions that assist with creating and saving plots, however each experiment functions as its own python script. The basic flow of each of these functions is as follows:

1. Establish constant, independent, and dependent variables
2. Run Training and Testing algorithms and using the Logger class to record data throughout their run-time
3. Compile a Pandas DataFrame using the recorded data
4. Clean the data and transform it into a long-data structure (sometimes completed at the same time as step 3)
5. Plot the DataFrame using Seaborn library

Experiments

Experiment 1: Weight Initialization



Objective

The goal of this experiment is to examine the effect that starting weights initalization has on the runtime of the linear regression model. The main details to look for if these weights have a major effect are in the convergence of the training and testing data sets. If the initial weights have a large impact, the weights would reach a convergence faster or slower than other trials of the experiment. You could also expect to find different values that each trial finds convergence on.

Methodology

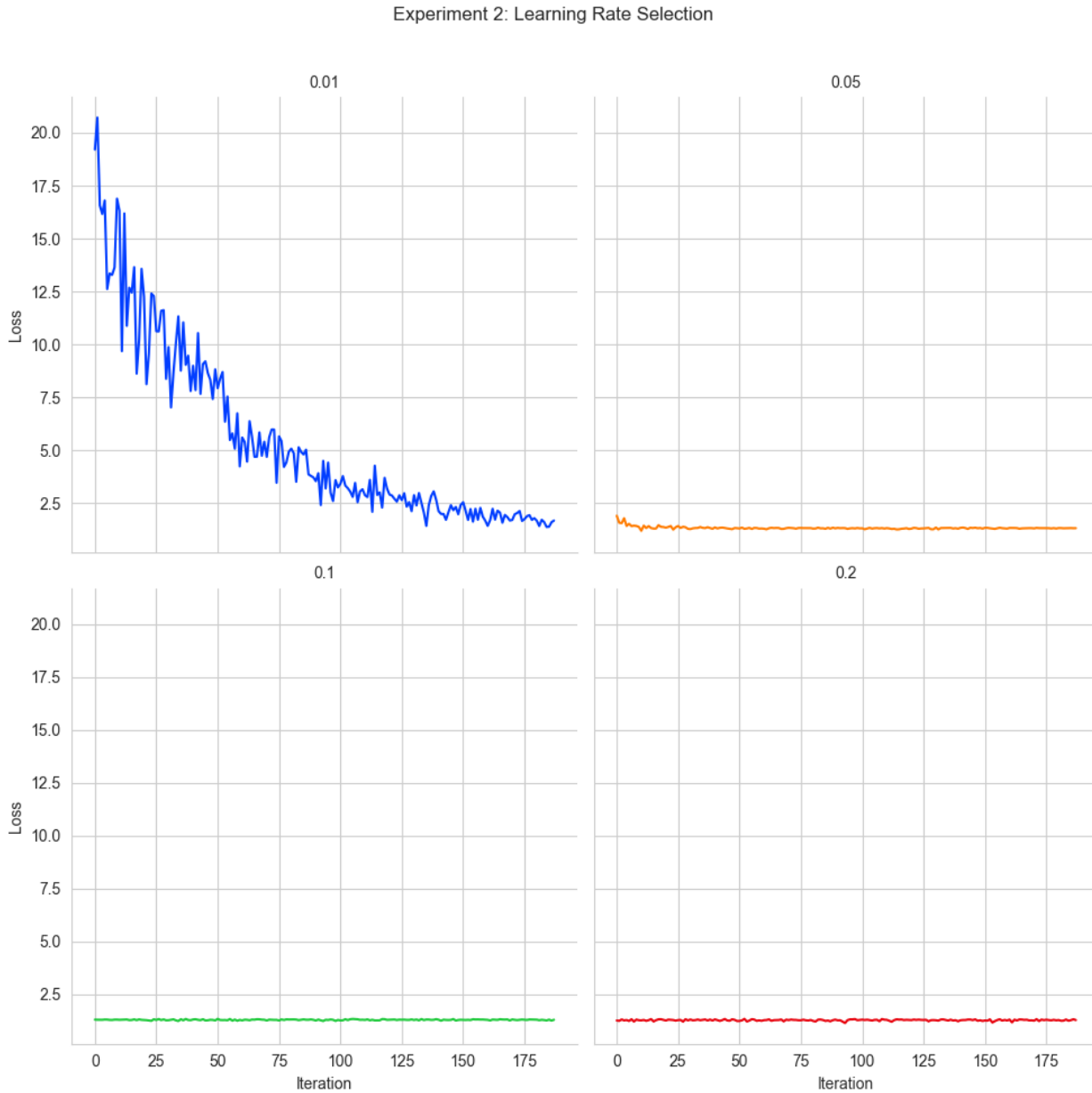
The methodology of this experiment was done in a different way than the others. Since I knew I wanted to graph the different than the other experiments, it was implemented without the use of the plotting utility function. This gave more freedom to implement the experiment with more descripitive naming instead of using more vauge names that traditionally work with more iterative workflows. The assignment requirements specify that the independent variables are setting the initial weights to all zeroes (being the

control group), a random normal distribution of numbers, and a uniform distribution of numbers, and examine the differences in the cost functions they produce.

Conclusion

The data largely supports that the value at which the weights start at does not have a major on the speed or value that convergence is found. This is rather different from past experiments that I have run with using various initial weights to find convergences. Typically when running this experiment I find that a large variety of numbers will find convergence in different local minima of the cost function. Gradient descent being a greedy algorithm (taking the immediate next action that leads to the closest local minimum), I would have expected that this experiment would have yeilded the same results. A reason that could explain why convergent behavor was so similar across all test trials could be the way that the initial weights are generated. Normal and Uniform distributions have a tendency to randomly generate numbers that are close to 0, so these minor changes in weights are not large enough to start encounting different local minima in the cost function.

Experiment 2: Learning Rate Selection



Objective

This experiment aims to study the impact that the hyper-parameter alpha (also known as the learning rate) has on the run-time of the linear regression model. Changing learning affects how much the weights used to in the calculation of the loss function change after each iteration. Larger values used for the learning rate means that greater changes can happen to the weights, and the algorithm will find the location of the convergence faster. However, if the learning rate is set too high, the algorithm can continuously overshoot finding the point of convergence, meaning that the algorithm will instead take longer to self-terminate.

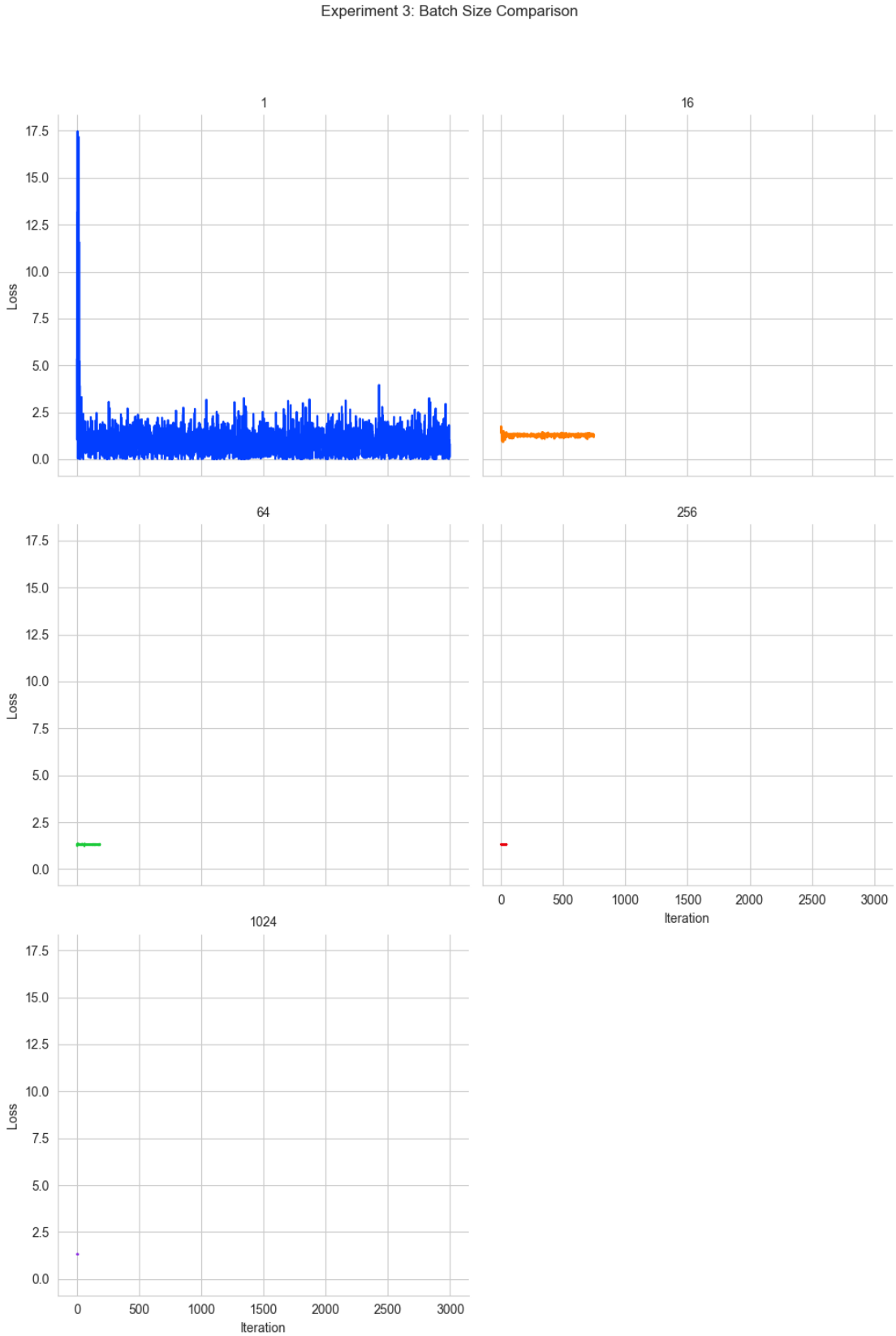
Methodology

For this experiment, the only changing hyperparameter, the learning rate, was set to values of 0.01, 0.05, 0.1, and 0.2. The standard parameter for learning rate is typically 0.01, so this means that all of the cases following 0.1, should have the behavior of a learning rate that is too big as described in the objectives of this experiment. The criteria for this assignment only require plots for the training data, so all plots featured for this section are generated with training data.

Conclusion

The data generated by this experiment does appear to follow in the description that was provided. It is easy to tell that as the learning rate increases, the faster that the value of the cost function finds the general value that convergence would be found. The 0.1 and 0.2 trials, in fact, found this point of convergence in their first few iterations. We can also see, however, that these trials took just as long as the 0.01 and 0.05 trials to terminate at a point of convergence, meaning that the higher learning rate did not have any actual benefit to the speed at which the algorithm performs at all.

Experiment 3: Batch Size Comparison



Objective

This experiment aims to evaluate how the size of the batches that are used in each iteration of the linear regression algorithm works in improving the speed at which the algorithm finds a point of convergence.

Working in batches is favored over working all samples at once, as the smaller iterations allow for more checks of if the algorithm has found convergence yet. A good medium of batch sizes will not unnecessarily speed resources calculating if a convergence has been reached yet and does not compute more data calculations than it must.

Methodology

Since the gradient descent algorithm is already capable of handling a variety of different batch sizes during its run-time, all that was required to be done was to input the desired sizes of batches and plot the data. The sizes selected for this experiment were: 1, 16, 64, 256, and 1024. The gradient descent algorithm was designed to process its data in batches of 64, so this group can act as the control for this experiment.

Flaws

During the creation of the plots for this experiment, I encountered an error where seaborn would not allow for the toggling of the shared x axis. I was unable to find a solution that yielded readable data. For this reason, the graphs for this experiment (and experiment 4) are a little difficult to fully examine and evaluate the more detailed aspects of their effects

Conclusion

I believe that these sets of graphs reveal the most amount of information in this report regarding how hyperparameters affect the rate at which convergence is found in linear regression models. It is blatantly obvious that the higher the batch size is, the fewer iterations that are required for finding the point of convergence. This is because you are processing more data at one time before concluding an iteration and

checking if a point of convergence was found. The 1024 trial group appears to have found convergence in under 100 iterations. This comes at a price, however, as this was also the most taxing of the algorithms to run on a computer due to the sheer size of the matrices and vectors required to perform functions on to complete a calculation. So, while the plots do lead to the conclusion that these convergences were found faster, these trails still took a long time per iteration compared to smaller batch sizes. Larger batch sizes also reduce the amount of noise present within the plots. As larger pools of samples are considered in each iteration, the cost function can average out some of the outliers found. This also means the inverse, that the test-trials executed with small batch sizes (such as batch size of 1) generate a substantially noisier plot that is difficult to examine. This also expends a lot of resources, as the algorithm must calculate if convergence was found after each iteration, which is not very likely to change in such small increments.

Experiment 4: Effect of Weight Decay Regularization

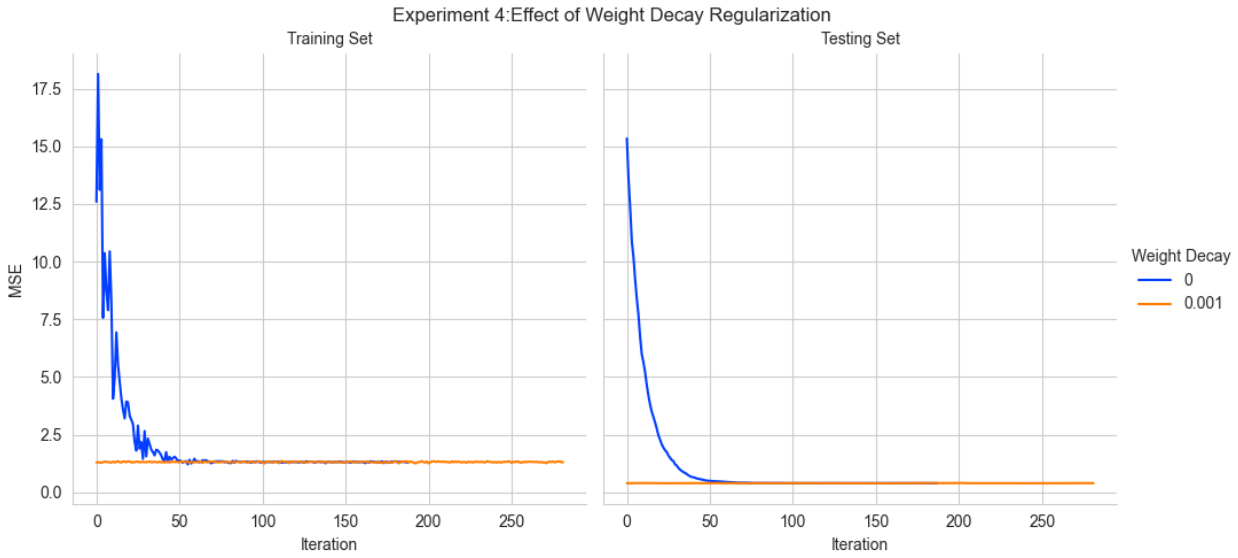


Figure 1

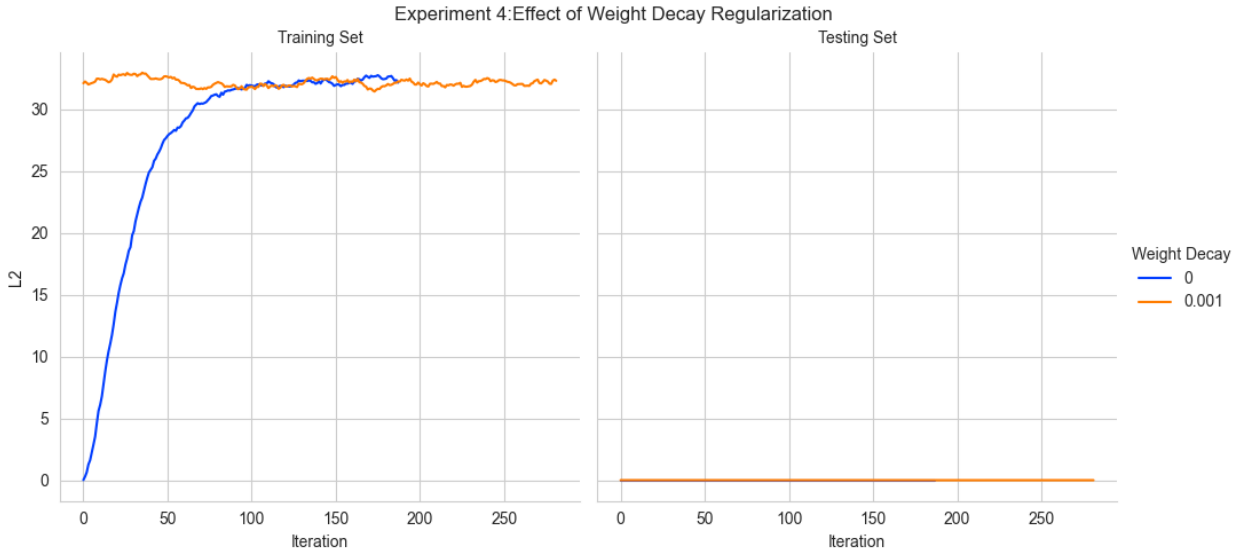


Figure 2

Objective

This experiment explores the effects that weight decay has on a linear regression algorithm. Weight decay acts in a way that always subtracts from the absolute value of the weights (pulling them back to 0 in the case of this experiment. In application, this allows the algorithm to ensure that it is not being overfit to the testing data, and that it will respond better to the testing data.

Methodology

This study features 2 groups, one group without weight decay (treated as the control), and one version with a small amount of weight decay. To understand the impacts of weight decay, the cost function to be broken down to the MSE error (figure 1) and the $||w||^2$ (figure 2). The MSE represents the amount that the estimated value generated by the sample features is off and is the main loss function the other experiments utilize. $||w||^2$ is a measure of how large the weights are and is used in weight decay to determine how much the weights should be penalized to bring them back down.

Flaws

An important note is the behavior that occurs on the Testing plot of figure 2. This is somewhat of an anomaly to me. I believe it might have to do something with how weights are not edited during testing, impacting some functioning of the calculation of this statistic. This being the only experiment that utilizes this data could also point to some sort of error in the creation or retrieval of this data.

Conclusion

It is evident through the MSE plots that the usage of weight decay does have some positive impact overall on the weights generated by the generated by the linear regression model it is employed in. The model using weight decay finds a point of convergence rather quickly, and I suspect it is due to the weight decay being too high that the algorithm struggles to find a point that it terminates on. We can also see that the MSE is overall very low for the weight decay model. There are some points during training that the cost

function is evaluated to be lower on the model without weight decay, which could be another sign that could point to the weight decay being set too high.